

```
# =====
# 📦 Import các thư viện cần thiết cho Data Mining
# =====

# Xử lý dữ liệu
import pandas as pd
import numpy as np

# Trực quan hóa dữ liệu
import matplotlib.pyplot as plt
import seaborn as sns

# Tiền xử lý và mô hình hóa
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.model_selection import train_test_split
from sklearn.cluster import KMeans
from sklearn.impute import SimpleImputer

# Kiểm định và thống kê
import scipy.stats as stats

# Cảnh báo hiển thị
import warnings
warnings.filterwarnings('ignore')

print("Import thư viện thành công!")
```

Import thư viện thành công!

```
from google.colab import files
uploaded = files.upload()

import pandas as pd
import io

df = pd.read_csv('listings_ready_change_01.csv')
df.head()
```

Choose Files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Saving listings_ready_change_01.csv to listings_ready_change_01.csv

	listing_id	host_id	host_name	host_since	host_response_rate	host_acceptance_rate	host_is_superhost	host_experience	prop
0	2708.0	3008.0	Chas.	16/09/2008 0:00	1.0	1.00	t	17.0	Priv
1	2732.0	3041.0	Louise-Diane	17/09/2008 0:00	1.0	0.50	f	17.0	Priv
2	6931.0	3008.0	Chas.	16/09/2008 0:00	1.0	1.00	t	17.0	Priv
3	7874.0	21700.0	Henry	15/06/2009 0:00	1.0	0.98	t	16.0	Priv
4	8021.0	22460.0	Sherree Strong	20/06/2009 0:00	1.0	1.00	f	16.0	Priv

5 rows × 27 columns

Chọn các biến đầu vào và biến đầu ra

```
# Chọn các biến đầu vào
features = [
    'host_response_rate',
    'host_acceptance_rate',
    'host_experience',
    'number_of_reviews',
    'reviews_per_month',
    'review_scores_rating'
]

# Biến mục tiêu
```

```
target = 'host_is_superhost' # 1 = superhost, 0 = host thường
```

Chia tập dữ liệu

```
from sklearn.model_selection import train_test_split

# Create X and y from the DataFrame
X = df[features]
y = df[target].apply(lambda x: 1 if x == 't' else 0)

# Chia tập 80% train - 20% test
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

print("Train size:", X_train.shape)
print("Test size:", X_test.shape)
```

```
Train size: (10217, 6)
Test size: (2555, 6)
```

```
def preprocess_features(X, y, features):
    # Điền giá trị thiếu bằng median
    imputer = SimpleImputer(strategy='median')
    X = pd.DataFrame(imputer.fit_transform(X), columns=features)

    return X, y, features
```

```
def train_rf(X_train, y_train):
    print("🌲 Đang train mô hình Random Forest tối ưu...")

    model = RandomForestClassifier(
        n_estimators=500,      # tăng số cây cho ổn định hơn
        max_depth=None,       # để mô hình tự tìm độ sâu tối ưu
        min_samples_split=5,  # giảm overfitting
        min_samples_leaf=2,   # giảm chia nhánh không cần thiết
        max_features='sqrt',  # chuẩn cho classification
        bootstrap=True,       # dùng bootstrap sampling
        random_state=42,
        n_jobs=-1             # chạy song song
    )

    model.fit(X_train, y_train)
    return model
```

```
# 9) MAIN PIPELINE - CHẠY TOÀN BỘ
# -----
if __name__ == "__main__":
    # Import các thư viện cần thiết cho pipeline
    from sklearn.ensemble import RandomForestClassifier
    from sklearn.metrics import confusion_matrix, classification_report, roc_auc_score, roc_curve

    # Dữ liệu (df, X, y, X_train, X_test, y_train, y_test, features)
    # đã được load và xử lý ở các cell trước.
    # Chúng ta sẽ dùng trực tiếp X_train, X_test, y_train, y_test và danh sách features.

    # 1. Tiền xử lý đặc trưng: xử lý giá trị thiếu trong X_train và X_test
    print("🌀 Đang tiền xử lý dữ liệu...")
    # preprocess_features sẽ xử lý và trả về DataFrame mới.
    # Dùng copy() để tránh SettingWithCopyWarning nếu hàm không xử lý nội bộ.
    X_train_processed, _, _ = preprocess_features(X_train.copy(), y_train, features)
    X_test_processed, _, _ = preprocess_features(X_test.copy(), y_test, features)

    # Lưu ý: y_train và y_test không thay đổi trong quá trình preprocess_features,
    # nên có thể dùng lại trực tiếp.

    # 2. Huấn luyện mô hình Random Forest
    model = train_rf(X_train_processed, y_train)

    # 3. Đánh giá mô hình
    preds, probs = evaluate(model, X_test_processed, y_test)
```

```
# 4. Hiển thị mức độ quan trọng của các biến
show_importance(model, features)

print("\n✅ DONE - MÔ HÌNH CHẠY THÀNH CÔNG!")
```

🌀 Đang tiền xử lý dữ liệu...
 🌲 Đang train mô hình Random Forest tối ưu...

📊 Confusion Matrix:
 [[886 522]
 [216 931]]

📋 Classification Report:

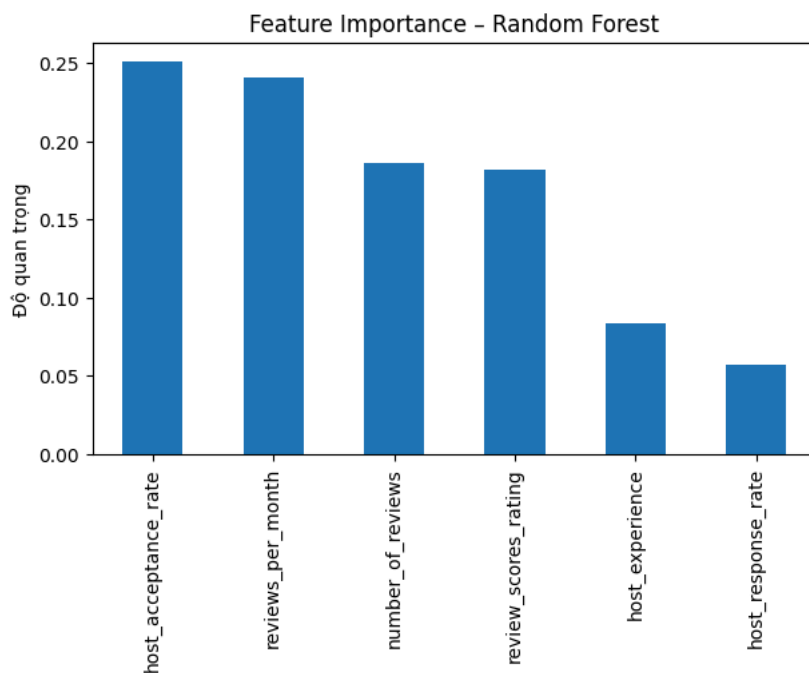
	precision	recall	f1-score	support
0	0.80	0.63	0.71	1408
1	0.64	0.81	0.72	1147
accuracy			0.71	2555
macro avg	0.72	0.72	0.71	2555
weighted avg	0.73	0.71	0.71	2555

📈 ROC AUC: 0.7925170405009114

🔥 Mức độ quan trọng của từng biến:

host_acceptance_rate	0.250429
reviews_per_month	0.240597
number_of_reviews	0.186060
review_scores_rating	0.182070
host_experience	0.083842
host_response_rate	0.057002

dtype: float64



✅ DONE - MÔ HÌNH CHẠY THÀNH CÔNG!

```
# 7) FEATURE IMPORTANCE
# -----
def show_importance(model, features):
    importances = pd.Series(model.feature_importances_, index=features).sort_values(ascending=False)

    print("\n🔥 Mức độ quan trọng của từng biến:")
    print(importances)

    # Vẽ biểu đồ
    plt.figure(figsize=(7,4))
    importances.plot(kind='bar')
    plt.title("Feature Importance - Random Forest")
```

```
plt.ylabel("Độ quan trọng")
plt.show()
```

Đánh giá mô hình

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, roc_auc_score, confusion_matrix, classification_report
from sklearn.metrics import roc_curve
import matplotlib.pyplot as plt

def evaluate_model(model, X_test, y_test):
    # Dự đoán
    y_pred = model.predict(X_test)
    y_prob = model.predict_proba(X_test)[:, 1]

    # Các chỉ số đánh giá
    acc = accuracy_score(y_test, y_pred)
    prec = precision_score(y_test, y_pred)
    rec = recall_score(y_test, y_pred)
    f1 = f1_score(y_test, y_pred)
    auc = roc_auc_score(y_test, y_prob)
    cm = confusion_matrix(y_test, y_pred)

    # In kết quả
    print("\n📊 KẾT QUẢ ĐÁNH GIÁ MÔ HÌNH")
    print(f"Accuracy:   {acc:.4f}")
    print(f"Precision:   {prec:.4f}")
    print(f"Recall:      {rec:.4f}")
    print(f"F1-score:    {f1:.4f}")
    print(f"ROC-AUC:     {auc:.4f}")

    print("\n📄 Confusion Matrix:")
    print(cm)

    print("\n📄 Classification Report:")
    print(classification_report(y_test, y_pred))

    # Vẽ ROC Curve
    fpr, tpr, _ = roc_curve(y_test, y_prob)
    plt.figure(figsize=(6,4))
    plt.plot(fpr, tpr, label=f"AUC = {auc:.4f}")
    plt.plot([0,1],[0,1], '--')
    plt.xlabel("False Positive Rate")
    plt.ylabel("True Positive Rate")
    plt.title("ROC Curve - Random Forest")
    plt.legend()
    plt.show()

model.fit(X_train, y_train)

evaluate_model(model, X_test, y_test)
```

KẾT QUẢ ĐÁNH GIÁ MÔ HÌNH

Accuracy: 0.8082
Precision: 0.7722
Recall: 0.8126
F1-score: 0.7918
ROC-AUC: 0.9019

Confusion Matrix:

```
[[1133 275]
 [ 215 932]]
```

Classification Report:

	precision	recall	f1-score	support
0	0.84	0.80	0.82	1408
1	0.77	0.81	0.79	1147
accuracy			0.81	2555
macro avg	0.81	0.81	0.81	2555
weighted avg	0.81	0.81	0.81	2555

